Monte Carlo Decomposition

Alejandra, Alfonso, Candela, Jaime, Marta and Rufus

November 2025

1 Introduction

Monte Carlo simulation is a numerical method used to approximate expectations and integrals through random sampling. Instead of attempting to solve complex mathematical expressions analytically, Monte Carlo methods rely on simulating a large number of random outcomes, treating them as data, and computing their empirical average. This approach is particularly useful in high-dimensional problems or in situations where closed-form solutions are unavailable or computationally impractical.
Many problems of interest can be expressed as expectations of the form

$$\mathbb{E}[X] = \int x f(x)\, dx,$$

where $f(x)$ is a probability density function. Monte Carlo methods approximate this expectation by generating independent and identically distributed samples $X_1, X_2, \ldots, X_N$ from the distribution and computing their sample mean,

$$\mathbb{E}[X] \approx \frac{1}{N} \sum_{i=1}^{N} X_i.$$

In words, Monte Carlo replaces an integral with an average over simulated data. As the number of samples increases, this average becomes a more accurate approximation of the true expected value. The basic idea behind Monte Carlo simulation is therefore simple: simulate many random outcomes of the same phenomenon, treat these outcomes as observations, and use their empirical mean to estimate the quantity of interest. This simplicity makes Monte Carlo methods extremely flexible and widely applicable across scientific disciplines.

The theoretical justification for Monte Carlo methods comes from fundamental results in probability theory, most notably the Law of Large Numbers and the Central Limit Theorem. These results explain why the Monte Carlo estimator converges to the correct value and how the approximation error behaves as the number of simulations increases. A detailed discussion of these statistical foundations is presented in the next section.

Historically, Monte Carlo methods were developed in the 1940s at the Los Alamos National Laboratory by Stanisław Ulam, John von Neumann, and other researchers working on the Manhattan Project. They were confronted with extremely complex problems in nuclear physics, such as modeling neutron diffusion and chain reactions, which could not be solved using traditional analytical

techniques. Ulam proposed using random sampling to simulate these processes, while von Neumann contributed to formalizing the method and implementing it on early computers. This marked the birth of Monte Carlo simulation as a practical numerical tool.

The name "Monte Carlo" was inspired by the famous casino city in Monaco, reflecting the method's reliance on randomness and chance. Initially applied in physics and engineering, Monte Carlo methods later spread to statistics, reliability engineering, and finance as computing power became widely available. Today, Monte Carlo simulation is a central numerical technique for evaluating complex probabilistic models, particularly in financial mathematics, where it plays a key role in option pricing and risk management.

2 Statistical Foundation

Montecarlo is fundamentally a statistical method, its accuracy and reliability depends entirely in two probabilistic results:

- The Law of Large numbers
- The Central Limit Theorem

Together, these results explain why Monte Carlo works, how fast it converges and what assumptions are necessary for it to be valid.

2.1 The Law of Large Numbers (LLN)

Consider a sequence of independent and identically distributed (i.i.d.) random variables

$$X_1, X_2, \ldots, X_N, \qquad \text{with } \mathbb{E}[X_i] = \mu.$$

The **sample mean** is given by

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i.$$

The **Law of Large Numbers** states that as the number of samples increases, the sample mean converges to the true mean:

$$\bar{X}_N \longrightarrow \mu \qquad \text{as } N \to \infty.$$

Intuition

Each X_i is random and can vary significantly, but as we average more and more samples, the randomness begins to *cancel out*. Even though individual outcomes fluctuate, the average becomes increasingly stable. This explains why, for example, the proportion of heads in many coin tosses gets closer to 0.5, even if the first few tosses may be irregular.

Role in Monte Carlo

Monte Carlo methods estimate an expectation such as

$$\mathbb{E}[X]$$

by computing the sample mean of many simulated values. LLN guarantees that

$$\widehat{\mu}_N = \bar{X}_N \to \mathbb{E}[X],$$

so the estimator becomes accurate when the number of simulations is large. Thus, LLN provides the theoretical guarantee that Monte Carlo *works*.

2.2 The Central Limit Theorem (CLT)

While LLN tells us that convergence occurs, it does not describe how *fast* the estimator approaches the true value or how the error behaves. The **Central Limit Theorem** fills this gap.
Under the same i.i.d. assumptions, the CLT states:

$$\sqrt{N}\,(\bar{X}_N - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2),$$

where σ^2 is the variance of X_i.

Interpretation

The CLT has two crucial implications:

- The Monte Carlo estimator is **centered around the truth**. This means the estimator does not systematically overestimate or underestimate the true value.
- The error behaves like **Gaussian noise**. Even if the original distribution of X_i is not Normal, the distribution of the estimator's error approaches a Normal distribution for large N.

Most importantly, the CLT implies that the Monte Carlo error decreases at the rate

$$O\left(\frac{1}{\sqrt{N}}\right).$$

Consequences

Monte Carlo converges slowly. To reduce the error by a factor of 2, we must increase the number of simulations by a factor of 4. To reduce it by a factor of 10, we need 100 times more simulations. This limitation motivates the development of variance reduction techniques used in practical Monte Carlo implementations.
Because the error is approximately Normal, the CLT also allows us to construct confidence intervals and measure the uncertainty of Monte Carlo estimates.

2.3 i.i.d. Assumptions and Their Importance

LLN and CLT both require that the samples be **independent** and **identically distributed**. This assumption is fundamental.

Independence If the samples influence one another, they do not provide new information. For example, if successive samples are positively correlated, then when one sample is high, the next one is also likely to be high. This makes the sample mean appear more stable than it truly is and underestimates the actual variability:

$$\text{Cov}(X_i, X_j) > 0 \quad \Rightarrow \quad \text{underestimated variance.}$$

This leads to overconfidence in the Monte Carlo estimate.

Identical Distribution All samples must come from the same distribution:

$$X_i \sim F \qquad \forall i.$$

If the distribution changes (for example, if one sample is drawn from one model and another from a different one), then the sample mean does not approximate any meaningful expectation.
Monte Carlo relies on i.i.d. samples to ensure that its theoretical guarantees apply.

2.4 Simulated Data vs. Real Data

A central concept in Monte Carlo is that the data we use are **simulated**, not observed.

Simulated Data (Monte Carlo)

- We generate samples from a known probability distribution chosen by the model.
- In finance, for example, stock prices might be simulated using the geometric Brownian motion model.
- Because we control the distribution, the i.i.d. assumptions hold exactly.
- LLN and CLT apply perfectly, giving Monte Carlo strong theoretical guarantees.

Real Data

- Collected from observations of the real world.
- The underlying distribution is unknown and may change over time.
- Data may violate independence due to autocorrelation or structural breaks.

Thus, Monte Carlo evaluates the behavior of a **model** rather than the real world. If the model is misspecified, Monte Carlo converges to the wrong value, even with millions of samples.

2.5 Summary

- The LLN guarantees that the Monte Carlo estimator converges to the true expected value.
- The CLT describes the shape and decay rate of the estimation error.
- The i.i.d. assumption is essential for both results.
- Monte Carlo uses synthetic, model-generated data, not real observations.
- These principles justify Monte Carlo as a numerical method for computing expectations, including those arising in option pricing and financial modeling.

3 Option Pricing

3.1 What Is an Option?

An **option** is a financial contract that gives the holder the *right*, but not the obligation, to buy or sell an asset at a specific price in the future.

The most common one, and the one we focus on, is the **European call option**. It is defined by:

- **Underlying asset**: usually a stock
- **Strike price** K: the future buy price written in the contract
- **Maturity** T: when the option expires
- **Payoff**:

$$\max(S_T - K, 0)$$

If the stock price S_T at time T is above K, the option is valuable. If not, it expires worthless.

3.2 Why Is Option Pricing Difficult?

The future price of a stock is **uncertain**, it could go up, down, or move unpredictably.

Traditional pricing models (before Black–Scholes) struggled because they had too many **unknown variables**, making exact pricing impossible.

The breakthrough idea in modern finance is:

> - An option's price is the **expected value** of its future payoff under the "risk-neutral" world of discounted cash flows.

But this "expected value" requires knowing **future stock price distributions** and that's where **Brownian motion** comes in.

3.3 Brownian Motion & Stock Prices

Why do we assume stocks follow Brownian motion?

The Black–Scholes model assumes stock prices follow a **Geometric Brownian Motion (GBM)**.

The idea is:

- Prices move randomly
- Prices can rise or fall continuously
- Bigger uncertainty $\rightarrow$ more volatility
- Markets evolve smoothly, not in sudden large jumps

In math, GBM is written as:

$$dS_t = \mu S_t \, dt + \sigma S_t \, dW_t$$

Where:

- S_t: stock price at time t
- μ: expected growth (drift)
- σ: volatility (uncertainty)
- W_t: standard **Brownian motion**, a continuous-time random walk
- dW_t: the random "shock" at each time step

Why Brownian motion makes sense:

- Stock returns over small intervals appear roughly normally distributed
- Prices cannot be negative, and GBM always stays positive
- Many small random influences (news, trades, macro events) aggregate into smooth randomness

This assumption is the foundation of **Monte Carlo simulation in finance**.

3.4 Risk-Neutral Pricing Principle

In modern financial mathematics, derivative prices are computed under the **risk-neutral measure**, a probability framework in which all assets are assumed to grow at the **risk-free rate** rather than their real-world expected returns.

The key result is:

> *Under the risk-neutral measure, the fair price of a derivative is the discounted expected value of its payoff.*

For a European call option, this means:

$$C_0 \;=\; e^{-rT}\,\mathbb{E}^{\mathbb{Q}}\big[(S_T - K)^+\big]\,.$$

This principle is extremely powerful: we do **not** need to predict whether the stock will rise or fall. We only need the *probabilistic distribution* of S_T under GBM, because the option price is simply the **discounted expectation** of its payoff.

Monte Carlo methods approximate this expectation numerically by simulating many possible values of S_T under the risk-neutral GBM model.

3.5 What Is Monte Carlo Simulation?

Monte Carlo simulation is a method that uses **random sampling** to estimate mathematical quantities that are too complex to solve analytically.

Monte Carlo Simulation: the simulation relies on repeated random sampling and statistical analysis to compute the results.

Why use Monte Carlo?

Because:

- Some equations have no exact formula
- Random systems can be understood by simulating many random outcomes
- The **Law of Large Numbers** guarantees that the average of many simulations converges to the true expected value

In option pricing, Monte Carlo does exactly that

- Simulate many possible stock price paths
- Compute payoff for each
- Average and discount them

3.6 How Monte Carlo Simulates Stock Prices

Using GBM assumptions, the exact formula to simulate future price S_T is:

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}\,Z\right]$$

Where:

- Z is a random draw from a standard normal distribution (mean 0, variance 1)
- r is the risk-free rate

This formula comes from solving the GBM stochastic differential equation.

3.7 Monte Carlo Method for Pricing a European Option

We want the present value of the expected payoff:

$$\text{Call Price} = e^{-rT}\,\mathbb{E}[\max(S_T - K, 0)]$$

Monte Carlo estimates this expectation by simulation.

Step 1: Choose number of simulations N

The more simulations, the more accurate the estimate. But too many simulations take too long.

Step 2: For each simulation, generate a random stock price

Use the GBM formula with a fresh random Z each time.
This matches the "random sampling" method described earlier.

Step 3: Compute payoff for each simulation

$$\text{payoff}_i = \max\Big(S_T^{(i)} - K, 0\Big)$$

Step 4: Average all payoffs

$$\overline{P} = \frac{1}{N}\sum_{i=1}^{N} \text{payoff}_i$$

This is the Law of Large Numbers at work. The Boyle (1977) paper used exactly this logic to price options by Monte Carlo.

Step 5: Discount the result to present value

$$C = e^{-rT}\,\overline{P}$$

Final Output

This value C is the Monte Carlo estimate of the option price.

4 Mathematical Background of the Application and Financial Meaning

4.1 Analytic Solution for a European Call

Under the Black–Scholes model, the price of a European call option with maturity T and strike K is given by the closed-form formula

$$C_0 = S_0 N(d_1) - K e^{-rT} N(d_2),$$

where

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

This formula results from modeling the underlying asset price as a geometric Brownian motion and constructing a risk-free hedging strategy in an arbitrage-free market. Under the risk-neutral measure, option values are discounted expectations:

$$C_0 = e^{-rT}\mathbb{E}[(S_T - K)^+]$$

The analytic Black–Scholes value serves as the benchmark for our Monte Carlo estimates, allowing us to check convergence and accuracy.

4.2 Interpretation of Model Parameters

Spot price S_0. The current market price of the underlying asset. A higher S_0 increases the probability of the option expiring in the money.

Strike price K. The fixed, predetermined price at which you buy the underlying asset.

Time to maturity T. The remaining lifespan of the option. Longer maturities increase uncertainty and usually increase option value due to greater optionality.

Risk-free rate r. Used to discount expected payoffs. Higher r increases call prices because future cash flows are discounted less.

Volatility σ. The most influential parameter. Higher volatility produces a wider distribution of possible future prices and increases the expected payoff of the option.

4.3 Asian Options

An Asian call option depends on the average price of the underlying asset:

$$\text{Payoff} = \max\left(\frac{1}{M}\sum_{j=1}^{M} S_{t_j} - K,\ 0\right).$$

Analytic formulas for arithmetic-average Asian options are generally unavailable because the average of a geometric Brownian motion does not have a closed-form distribution. Monte Carlo simulation is therefore particularly useful, since it can generate full paths and compute averages directly.

4.4 Financial Meaning and Practical Importance

European and Asian options are widely used for hedging, risk management, and structured financial products. European calls hedge against upward price movements, while Asian options reduce sensitivity to short-term volatility spikes and are common in commodities and foreign exchange markets.
Accurate numerical pricing is essential for valuation, risk reporting, and regulatory requirements. Monte Carlo methods are especially valuable because they handle path-dependent payoffs naturally, work under a wide range of underlying models, and remain effective when closed-form solutions do not exist.

5 Monte Carlo in Modern Finance

5.1 Background

Monte Carlo methods were introduced into finance in the 1970s as a practical numerical tool for option pricing, most notably through the work of Boyle (1977). At that time, analytical solutions such

as the Black–Scholes formula were available only for a limited class of derivatives. As financial products grew more complex, especially with the introduction of path-dependent and high-dimensional payoffs, traditional closed-form pricing formulas and low-dimensional numerical methods became inadequate.

The increasing availability of computational power from the 1980s onward allowed Monte Carlo simulations to become feasible in financial institutions. Since then, Monte Carlo has become a standard technique in pricing, risk management, and regulatory frameworks. Its adoption is largely due to its flexibility and its strong theoretical foundation in probability theory, particularly the Law of Large Numbers and the Central Limit Theorem.

5.2 Why is Monte Carlo still used?

Despite the development of many alternative numerical methods, Monte Carlo simulation remains widely used in modern finance for several key reasons:

- **Flexibility.** Monte Carlo can handle a very wide range of financial products, including complex and path-dependent payoffs such as Asian options, barrier options, and structured products. As long as the underlying dynamics can be simulated, the payoff can be evaluated.
- **High-dimensional scaling.** Unlike grid-based methods, Monte Carlo does not suffer severely from the curse of dimensionality. Its convergence rate depends primarily on the number of simulations rather than on the dimension of the problem, making it suitable for multi-factor models and long time horizons.
- **Model-agnostic nature.** Monte Carlo works under many different underlying models, not only the Black–Scholes framework. It can be applied to stochastic volatility models, jump-diffusion models, and other extensions without major changes to the methodology.
- **LLN/CLT interpretability.** Monte Carlo estimates have a clear statistical interpretation. The Law of Large Numbers guarantees convergence to the true expected value, while the Central Limit Theorem provides a natural way to estimate the error, which decreases at rate $1/\sqrt{N}$. This allows practitioners to quantify uncertainty through confidence intervals.
- **Industry infrastructure.** Monte Carlo is deeply embedded in financial industry systems. It is the standard approach used in risk engines for Value at Risk (VaR), CVaR, XVA calculations, and regulatory stress testing. Existing infrastructure and validation procedures further reinforce its continued use.

5.3 Neural networks and surrogate models

A recent development in computational finance is the use of neural networks as surrogate models for pricing. In this framework, Monte Carlo simulations are first used to generate a large dataset of model parameters and corresponding prices. A neural network is then trained to approximate the pricing function. Once trained, the neural network can produce prices almost instantaneously. This approach is particularly useful for real-time pricing and calibration tasks, where thousands of evaluations are required. However, neural networks do not replace Monte Carlo as a reference method. Their accuracy depends on the quality and coverage of the training data, and they may

perform poorly outside the region where they were trained. For this reason, Monte Carlo remains the benchmark method, while neural networks act as fast approximations built on top of Monte Carlo-generated data.

In conclusion, Monte Carlo simulation continues to play a central role in modern finance due to its flexibility, scalability, and solid probabilistic foundation. While alternative numerical methods and neural-network surrogates can offer efficiency gains in specific contexts, Monte Carlo remains the reference framework against which other methods are validated.

6 Implementation and Numerical Experiments

6.1 Purpose of this section

This section documents the implementation and numerical experiments we carried out for our project. The goal is to show step by step, how Monte Carlo simulations can be used to price options and how the numerical results connect to the theory (see Raychaudhuri; An and Su). All the code is publicly available in the following repo: `https://github.com/apayanieu/montecarlo_nm`.
Throughout all experiments we use the same basic Monte Carlo estimators. Given discounted payoffs $X_1, \ldots, X_N$, we estimate the option price by

$$\widehat{C}_N = \frac{1}{N}\sum_{i=1}^{N} X_i, \qquad \widehat{\mathrm{SE}}(\widehat{C}_N) = \frac{s_X}{\sqrt{N}}, \qquad \mathrm{err}_N = \left|\widehat{C}_N - C_{\mathrm{ref}}\right|,$$

where s_X is the sample standard deviation and C_{ref} is the analytic Black–Scholes price. Here, we will use and prove the statistical foundations of Monte Carlo (LLN, CLT and synthetic data), which were explained in other sections of this report.
For our simulations, the underlying asset is modelled under the risk-neutral measure as a geometric Brownian motion

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t,$$

with baseline parameters for all Black Scholes experiments:

$$S_0 = 100, \quad K = 100, \quad r = 0.05, \quad \sigma = 0.20, \quad T = 1.$$

Here, we will work under the assumption of constant volatility σ, which is deliberately unrealistic. However, it will help us link the Monte Carlo estimator and the closed formula for European Option pricing as transparent as possible (An and Su 30–32).

6.2 Random-number set-up

Our project's pipeline uses a fixed seed for every experiment, guaranteeing reproducibility, that is, every time the experiments are rerun, the same sequence of synthetic inputs and outputs are obtained.
As a first sanity check, we draw a large sample form a normal distribution, which yielded the following graph: As a first sanity check, we draw a large sample from $\mathcal{N}(0, 1)$.

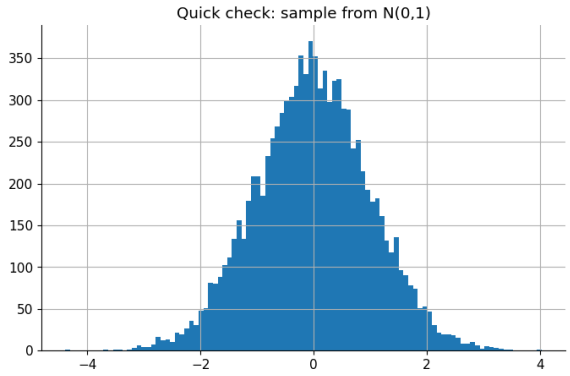


Figure 1: Sanity Check N(0,1)

This quick inspection mirrors Raychaudhuri's tutorial, which tells us to get samples from a known distribution. Moreover, this confirms that the pseudo-random generator will give us a reasonable approximation of Gaussian noise, which will later help us upon noticing the LLN/CLT arguments underlying montecarlo are indeed true.

6.3 Black-Scholes benchmark

After checking the basic random input, we fix the Black–Scholes parameters to the ones defined above, and we define the grid of simulation sizes N that will be reused in all later convergence plots: $\{10^3,\ 2\cdot 10^3,\ 5\cdot 10^3,\ 10^4,\ 2\cdot 10^4,\ 5\cdot 10^4,\ 10^5,\ 2\cdot 10^5,\ 5\cdot 10^5,\ 10^6\}$
Keeping this grid fix, make the experiments consistent, yielding intelligible results across the project. We then implement the closed-form Black–Scholes price for a European call.

$$C_{\mathrm{BS}}(S_0, K, r, \sigma, T) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2),$$

Evaluating this formula with the baseline parameters gives us

$$C_{\mathrm{BS}} = 10.450584, \qquad d_1 = 0.35, \qquad d_2 = 0.15.$$

Where 10.45 will by our analytic benchmark for our European Call experiments. The guiding idea is simple: if our simulation code is correct, given the statistical assumptions, the Monte Carlo estimator $\widehat{C}_N$ should converge numerically to this benchmark as the number of simulation grows.

6.4 European call pricing and convergence studies

Plain Monte Carlo estimator

For our next step of the experiment, we implement a function named `mc_european_call_price` that yields an estimate of the discounted payoff using Monte Carlo Simulations

$$X_i = e^{-rT}\big(S_T^{(i)} - K\big)^+, \qquad S_T^{(i)} = S_0\exp\big((r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z_i\big), \quad Z_i \sim \mathcal{N}(0,1).$$

Explaining it sequentially, we first generate standard normal shocks Z_i, which are mapped into terminal prices $S_T^{(i)}$ through the GBM solution, and finally converted into discounted call payoffs X_i. The Monte Carlo estimator $\widehat{C}_N$ is then just the average of these payoffs.
Three histograms make this pipeline explicit:

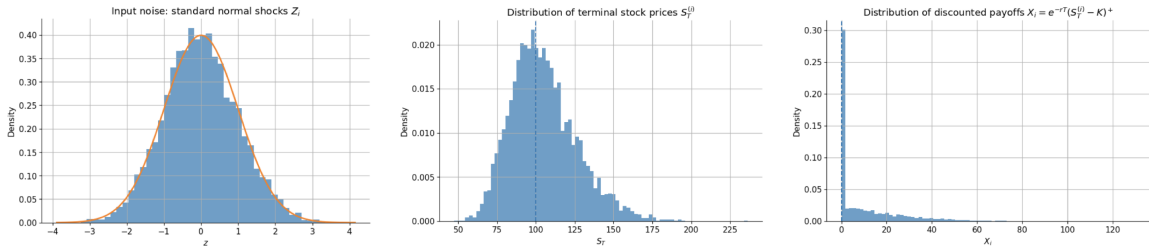


Figure 2: Monte Carlo Estimator for European Call Pipeline

- *"Input noise: standard normal shocks Z_i"* shows the Gaussian input that drives the GBM.
- *"Distribution of terminal stock prices $S_T^{(i)}$"* shows the lognormal distribution of terminal prices generated by the model.
- *"Distribution of discounted payoffs $X_i = e^{-rT}(S_T^{(i)} - K)^+$"* shows a large mass at zero with a long right tail, matching the usual intuition that most paths finish out of the money.

Convergence in the number of paths N

To study convergence we generated a million samples. For each N in the pre-defined grid of sample sizes, the estimate $\widehat{C}_N$ and its standard error are computed using only the first N samples. This produces, for each N, a point estimate and a statistical error bar based on the empirical variance of the payoffs.
As a reference, our largest simulation yields $N = 10^6$:

$$\widehat{C}_{10^6} = 10.4532, \qquad \widehat{\mathrm{SE}}(\widehat{C}_{10^6}) \approx 0.0147.$$

Approximating the analytic benchmark, $C_{\mathrm{BS}} = 10.4506$ with a small deviation: about 0.18 standard errors. This goes in line with our predictions based on CLT/LLN.
The behaviour of $\widehat{C}_N$ and its error is visualised in four plots:

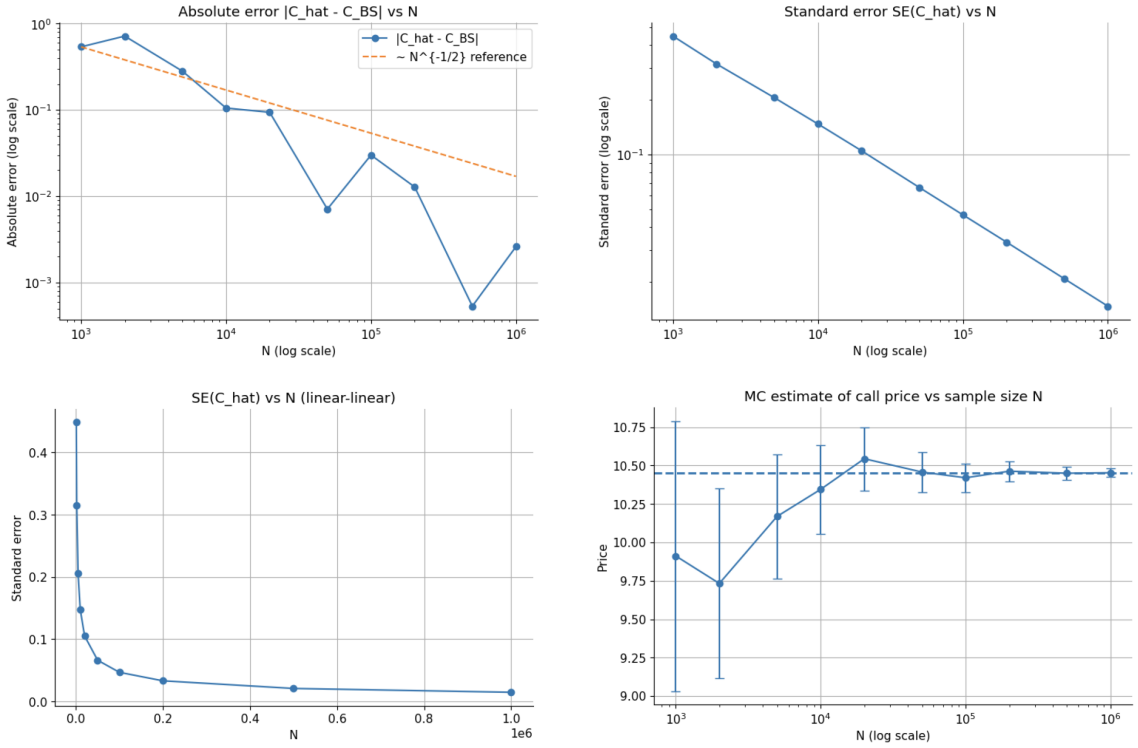


Figure 3: Convergence of Monte Carlo on European options

Across all of them, the same picture emerges: as the number of simulations N increases, the Monte Carlo estimator converges to the benchmark and the error decays as $N^{-1/2}$–confirming the LLN/CLT behaviour of Monte Carlo Simulations.

6.5 On Asian Calls

Path simulation and payoff distribution

To move beyond payoffs depending only on the terminal price, where Black-Scholes closed formula is often more effective under constant volatility assumptions, we introduce the arithmetic Asian Call, a path-dependent contract whose payoff depends on the average price of the underlying over time. Differing from European Calls, these are more complex, and Monte Carlo integration makes more sense applied to this type of financial instruments due to its high dimensional nature. Its payoff is defined by:

$$X_i^{\text{Asian}} = e^{-rT}\left(\frac{1}{M}\sum_{j=1}^{M} S_{t_j}^{(i)} - K\right)^+ .$$

For this experiment, first we simulate N GBM paths on a discretised grid of $M = 252$, trading days over a year, using a function `simulate_gbm_paths`. Then, a function, `price_asian_call_from_paths`,

will take these paths, and compute the average along each one, and evaluate the discounted payoff. After this, the option price will be estimated by an average of these path-dependent payoffs. Unlike European payoffs, here the distributions will be tighter, reflecting a reduction of variance due to the effect of averaging along the path. The behaviour of our simulations is consistent with Zhang's report on pricing Asian options with Monte Carlo, and it is shown in these two histograms below (Zhang 24–27):

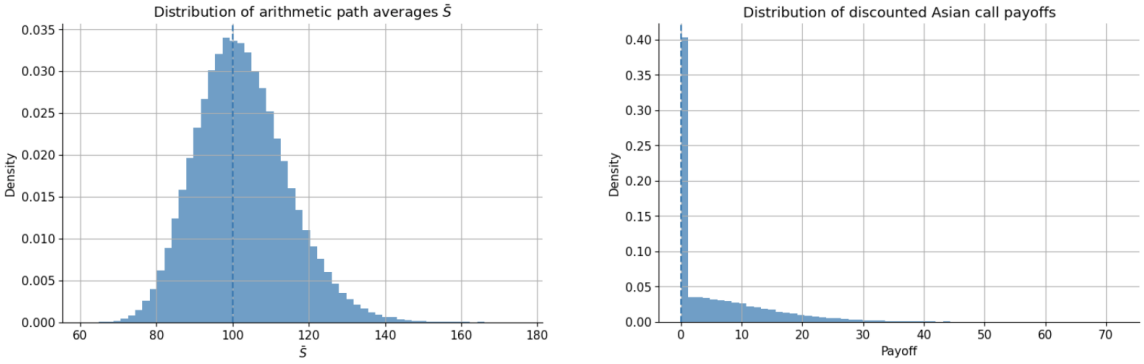


Figure 4: Distributions of path averages and payoffs

Path and payoff structure

For our next section, we need to mention that we reduced Monte Carlo variance using a control variate explained on Zhang's report (Zhang 18-23), which is constructed from a closed form expression for geometric Asian calls.

Before we jumped into the convergence study, first we wanted to illustrate the shape of risk of Asian options. For this, we created a graph showing a small bundle of sample paths (e.g. the first 20 trajectories out of the full Monte Carlo sample) on $[0, T]$. As it can be seen in the graph, the individual GBM paths can be quite volatile, but their averages, which is what we want, will be more stable.

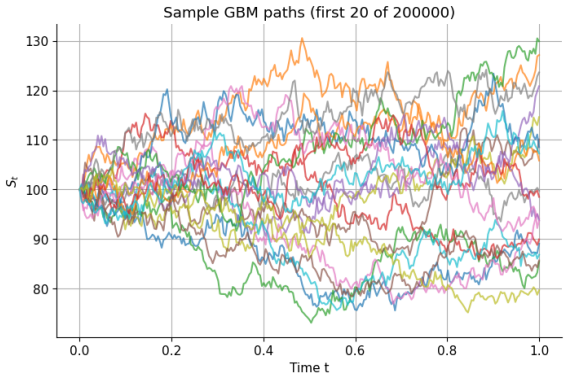


Figure 5: 20 GBM Paths

Convergence in the number of paths N

Finally, as well as in the European call example, we study the convergence of the Monte Carlo estimator for the Asian call price as the number of simulated paths increases, while keeping the time fixed at $M = 252$. We will use the same grid as in the past experiments, and we will compute a "reference" Asian price using a simulation over 1 million paths, which we will use as a high-precision metric for the true price.
For each N in the grid we then:

1. simulate N GBM paths with the same parameters and $M = 252$,
2. compute the corresponding Monte Carlo estimator $\widehat{C}_N^{\text{Asian}}$ and its standard error,
3. record the absolute error $|\widehat{C}_N^{\text{Asian}} - \widehat{C}_{N_{\text{ref}}}^{\text{Asian}}|$.

As we saw with European Calls, Monte Carlo will eventually decrease the error, and will stabilize around the reference price in the estimation as N increases. This also confirms the predictions of the Central limit Theorem for Monte Carlo estimators. This is both shown in the two graphs below: The first plot displays $\widehat{C}_N^{\text{Asian}}$ as a function of N on a logarithmic x-axis, with a horizontal line at the reference value $\widehat{C}_{N_{\text{ref}}}^{\text{Asian}}$. Here, we can see that as our number of simulations N increases, our Monte Carlo estimates will converge to the reference price the error will shrink in proportion to $N^{-1/2}$.
The second plot displays the absolute error $|\widehat{C}_N^{\text{Asian}} - \widehat{C}_{N_{\text{ref}}}^{\text{Asian}}|$ against N on a log–log scale, with a line that shows us the $N^{-1/2}$ rate. The empirical error curve is really close to the theoretical rate across the entire range of N (since this are very small errors), confirming that:

$$|\widehat{C}_N^{\text{Asian}} - C^{\text{Asian}}| \approx \mathcal{O}_{\mathbb{P}}(N^{-1/2}),$$

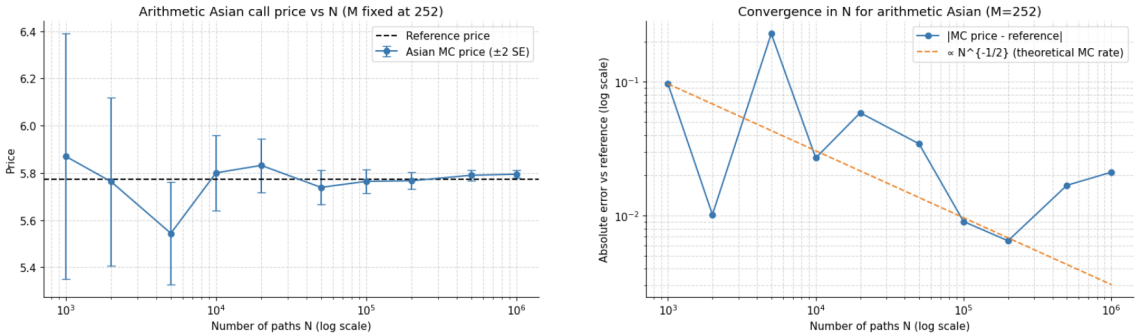


Figure 6: Convergence of Monte Carlo for Asian calls

6.6 Modeling with real-market data

Data collection and log-return statistics

After experimenting Monte Carlo machinery in a purely synthetical world, we wanted to see how well this model faced real data. For this, we chose to work with the SPY ETF, which tracks the S&P 500. Using data from yahoo finance, we downloaded the SPY prices over a five year-window and computed daily log returns.
From these log-returns, we estimated the historical mean and volatility:

$$\widehat{\mu}_{\text{daily}} = 5.54 \times 10^{-4}, \qquad \widehat{\sigma}_{\text{daily}} \approx 0.01076,$$

$$\widehat{\mu}_{\text{annual}} \approx 0.140, \qquad \widehat{\sigma}_{\text{annual}} \equiv \sigma_{\text{hist}} \approx 0.1708.$$

Historical-volatility repricing

Using the last SPY price in the sample as spot, an at-the-money strike $K_{\text{hist}} = S_{0,\text{hist}} = 681.76$, maturity $T_{\text{hist}} = 0.5$ years and a risk-free rate $r_{\text{hist}} = 3\%$, we computed Black-Scholes and Monte Carlo prices with historical volatility:

$$C_{\text{BS}}(\sigma_{\text{hist}}) = 37.9187, \qquad C_{\text{MC}}(\sigma_{\text{hist}}) = 37.8612, \qquad \widehat{\text{SE}}(C_{\text{MC}}) = 0.1226,$$

using $N = 200\,000$ paths.
Since the two values are statistically indistinguishable–the relative difference is about -0.15%, well within one standard error, we again confirm that when volatility is fixed, Monte Carlo and Black-Scholes agree numerically as predicted in our first section.

Market option price and implied volatility

The next step is to take the model and try it against a real option price. A SPY call with maturity close to T_{hist} is retrieved, yielding

$$S_{0,\text{mkt}} = 681.76, \quad K_{\text{mkt}} = 680.00, \quad T_{\text{mkt}} \approx 0.515, \quad C_{\text{mkt}} = 38.98.$$

Using historical volatility σ_{hist} and market maturity T_{mkt}, we compute the model prices:

$$C_{\text{BS}}^{\text{mkt}}(\sigma_{\text{hist}}) = 38.6679, \qquad C_{\text{MC}}^{\text{mkt}}(\sigma_{\text{hist}}) = 38.7136, \qquad \widehat{\text{SE}} \approx 0.1269.$$

Once again, our Black-Scholes and Monte Carlo prices are internally consistent, but they both underprice the market option by roughly 0.3 units.
Finally, we made a summary table for the comparison of three similar scenarios: pricing with historical volatility at historical maturity, pricing with historical volatility at market maturity, and the observed market price using the implied volatility inferred from the option quote. Our implementation yielded:

Table 1: Historical-volatility model vs SPY market price

Scenario	S_0	K	T (years)	σ	C_{BS}	C_{MC}
Model @ T_{hist} (σ_{hist})	681.76	681.76	0.5000	0.1708	37.9187	37.8612
Model @ T_{mkt} (σ_{hist})	681.76	680.00	0.5151	0.1708	38.6679	38.7136
Market (implied σ_{impl})	681.76	680.00	0.5151	0.1683	38.9800	–

In addition, we also computed the differences $C_{\text{BS}} - C_{\text{mkt}}$ and $C_{\text{MC}} - C_{\text{mkt}}$. At historical maturity our model underprices the option by about -1.06 units (-2.7%); at market maturity the gap shrinks to roughly -0.3.
This behaviour fits what we thought in the beggining, and confirms the story of the literature: real-world volatility is not constant, and constant volatility will not yield consistent results (see An and Su; Kunsági-Máté et al.).

6.7 Summary table and interpretation

To summarise the core numerical outcomes, Table 2 collects the main experiments in a single place.

Table 2: Summary of core Monte Carlo results

Experiment	Reference value C_{ref}	MC estimate $\widehat{C}_N$	MC SE	Relative gap
European call (synthetic, constant σ)	10.4506	10.4532	0.0147	0.02% (vs analytic)
Asian call (arithmetic, $M = 252$)	5.7729	5.7820	0.0005	0.16% (vs reference MC)
SPY ATM call (model with σ_{hist})	37.9187	37.8612	0.1226	-2.72% (model vs market)

When looking upon the European and Asian calls under constant-volatility assumptions, for large N the value estimated is effectively unbiased and its error behaves like $N^{-1/2}$ (Raychaudhuri; Zhang). The last rows highlights the ingenuity of implying constant volatility in market modeling: Black Scholes, and Monte Carlo with very large N will not be able to capture a close approximation to the price, which is needed for high liquid assets as the SPY ETF.

6.8 Possible extensions to this project

- **Better dynamics** Replace constant volatility with stochastic or local-volatility models. This is standard in modern practice and would reduce the structural error in market applications (An and Su 33–34).
- **Neural-network surrogates.** Use Monte Carlo to generate training data (parameters → price) and fit a neural network to approximate this mapping, following the "Deep Weighted Monte Carlo" approach of Kunsági-Máté et al.

7 References

- An, Nier, and Bocheng Su. "Pricing Options and Monte Carlo Method: A Literature Review." *BCP Business & Management: FIBA 2023*, vol. 44, 2023, pp. 29–35.
- Kunsági-Máté, Sándor, et al. "Deep Weighted Monte Carlo: A Hybrid Option Pricing Framework Using Neural Networks." Eötvös Loránd University, 2022.
- Raychaudhuri, Samik. "Introduction to Monte Carlo Simulation." *Proceedings of the Winter Simulation Conference*, 2008.
- Zhang, Hongbin. *Pricing Asian Options Using Monte Carlo Methods*. Uppsala University, 2009.
- Boyle, P. P. "Options: A Monte Carlo Approach." *Journal of Financial Economics*, 1977.
- Glasserman, Paul. *Monte Carlo Methods in Financial Engineering*. Springer, 2004.